Pic2Geom: A Fast Rendering Algorithm for Low-Poly Geometric Art

Ruisheng Ng, Lai-Kuan Wong, and John See

Faculty of Computing and Informatics, Multimedia University, Persiaran Multimedia, 63100 Cyberjaya, Selangor, Malaysia

Abstract. Low poly rendering has always been a popular form of art in the art and design community, where the artist draws each polygon (usually triangle) on the image individually. Several state-of-the-art methods were proposed to overcome the laborious process of manual low poly rendering, by rendering the low poly shapes automatically. However, results generated by the aforementioned methods were either not visually pleasing or the algorithms are computationally slow. In this paper, we present Pic2Geom, a fast algorithm that generates low poly geometric art with adequate quality, at low computational cost. The proposed algorithm utilizes edge detection, saliency detection and face detection to generate a set of seed points, which is then used by Delaunay triangulation to generate the low poly abstraction. Comparison with state-of-the-arts approaches demonstrate the efficiency and effectiveness of our algorithm in producing a quality geometric abstraction of a given photograph.

Keywords: Image abstraction, low poly rendering, geometric abstraction

1 Introduction

Geometric abstraction is a fork from image abstraction where it aims to discard unnecessary information from the image to create an abstract image, with the difference that it uses geometric shapes to represent its abstraction. Many works have been done on image abstraction, such as pixelated image abstraction by Gerstner et al [11] and Kuwahara filtering abstraction by Kyprianidis et al [10], but very few of these works focus on geometric abstraction.

Zhang et al [2] proposed a real-time low-poly style image and video processing algorithm. To achieve real-time low-poly video processing, the processing speed has to be minimal as a typical video clip contains around 15 to 30 frames per second. Their algorithm achieved an impressive computational speed that hovers around 0.1 to 0.2 seconds. However, the abstraction produced by their approach lacks perceptual accuracy as photo subjects are not being identified and considered during the abstraction process.

Gai et al [1] proposed a fully automated low poly rendering method for images. Their approach aims to create high quality low-poly art by constraining edge features and incorporating custom color post-processing to reduce the

Fig. 1. Results of Pic2Geom.

zigzag artifacts produced during the abstraction process. Besides, to produce a perceptually accurate abstraction, it utilizes salient region detection to identify the photo subjects in an image and use more triangles to represent the photo subjects. The results produced by their algorithm are visually appealing, with accurate abstraction over a wide variety of images. However, this approach lacks computational efficiency. Compared to Zhang et al [2] approach, the computational time of Gai et al [1] algorithm is almost 20 times higher.

In this paper, we present an efficient algorithm, Pic2Geom, that aims to produce perceptually accurate geometric abstraction with computational time that is comparable to Zhang et al [2] algorithm. In a nutshell, our proposed approach first identify the photo subjects. Based on the identified photo subjects, it generates a set of seed points that are then used to produce triangle-based, low poly geometric art. Fig.1 illustrates example results of Pic2Geom. Empirical evaluations conducted illustrate the effectiveness of Pic2Geom in producing high quality abstraction at low computation time.

2 Proposed Method

Fig.2 illustrates the overview of Pic2Geom. Given an input image, Pic2Geom first identify the important area of the image using a combination of saliency, edge and face detection techniques. It then generates a set of seed points, $P=\{P_{\mathrm{s}}, P_{\mathrm{e}}, P_{\mathrm{f}}\}$ to represent the important area, where P_{s}, P_{e} and P_{f} are points generated based on the detected salient regions, edges and faces respectively. These seed points are then used by Delaunay triangulation [7] to generate a set of triangle meshes. To produce the final geometric abstraction of the image, each triangle mesh will be colored with the average color of the pixels within the mesh as shown in the output image in Fig.2.

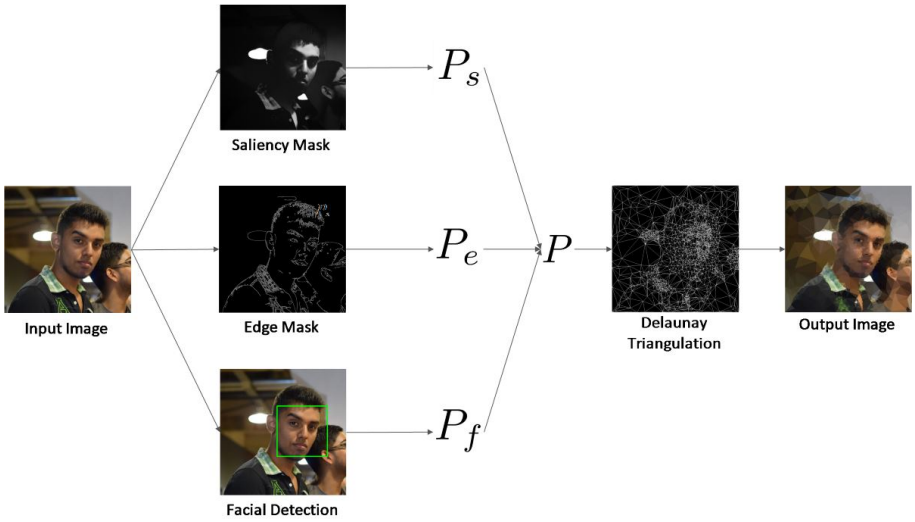

Fig. 2. Overview of Pic2Geom Algorithm. P_s is the salient points. P_e is the edge points. P_f is the facial points. P is the sum of salient points, edge points, and facial points.

To simplify the user input in performing the abstraction, we utilize a single abstraction parameter, N_t to control the amount of triangles being generated on the image. This N_t parameter is used by the salient detector, edge detector, and facial detector to determine the number of seed points to be generated by each detector. A higher N_t value increases the number of triangles generated, leading to lower level of abstraction of the image and vice versa. In our experiments, we set N_t to 200, as it produces an image that is sufficiently abstract whilst maintaining its detail. Fig.3 illustrates the abstraction results produced using different N_t values.

2.1 Saliency Detection

We adopt Perazzi's saliency model [3] as our salient detector because it has relatively good detection performance and is computationally efficient. From the saliency mask produced by the salient detector, we observed that the salient regions are often very focused and much smaller than the non-salient regions. Thus, based on the assumption that visually salient regions are smaller than the less salient regions, our algorithm segments the saliency mask into four regions; S_a, S_b, S_c, and S_d, with the salient values of each segment in the range of 0 to 63, 64 to 127, 128 to 191, and 192 to 255 respectively. Example of a segmented salient mask is shown in Fig.4(b). We then allocate N_t seed points to each segmented region. Thus, the number of salient seed points for an image, $|P_\mathrm{s}|$ is 4 x N_t. This results in higher density triangles being produced in the more salient regions.

Fig. 3. Comparison of results generated using different N_t values. **(top left)** $N_t = 100$, **(top right)** $N_t = 200$, **(bottom left)** $N_t = 300$, **(bottom right)** $N_t = 400$

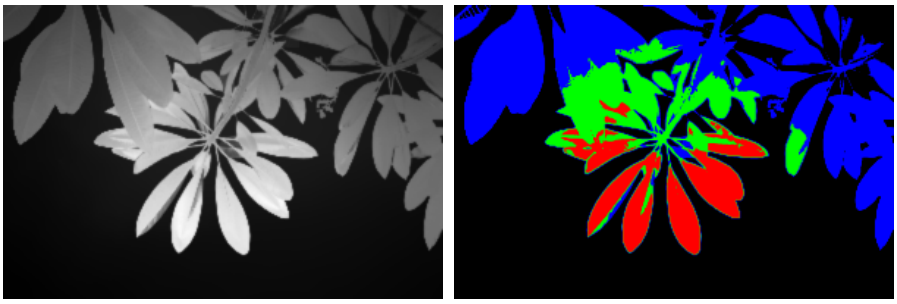

Fig. 4. **(left)** Salient mask. **(right)** Segmented regions based on saliency values in ascending order: black (least salient region), blue, green, red (most salient region).

2.2 Edge Detection

Prior to performing edge detection, we apply bilateral filter [4] to filter out any excessive noise from the input image. We then apply Canny edge detector [5] to detect the edges. Next, a set of seed points will be generated along the detected edges. To generate points on the edges, we first compute $xGap$ and $yGap$ using the following equations,

Fig. 5. Use of different N_t values for edge detection. **(left)** N_t=200 **(right)** N_t=350

$$xGap = \frac{I_w}{N_t} \tag{1}$$

$$yGap = \frac{I_h}{N_t} \tag{2}$$

where $xGap$ is the gap between two consecutive edge points on the x-axis; $yGap$ is the gap between two consecutive points on the y-axis; I_w is the width of the image in pixels; and I_h is the height of the image in pixels. We then scan through the image using the computed $xGap$ and $yGap$ values and insert a seed point if the selected pixel lies on an edge. P_e is the set of all the edge points. Fig.5 illustrates the seed points generated using different N_t values.

As the values of $xGap$ and $yGap$ are proportional to the width and height of the image respectively, the number of seed points generated is invariant to the image resolution and our algorithm is scalable for different image resolution. Notably, our algorithm would produce more dense seed points on vertical and horizontal edges compared to diagonal edges but this inconsistency would not affect the visual abstraction result substantially.

2.3 Face Detection

Although the use of both saliency and edge detection are sufficient to create aesthetically pleasing geometric art for many images, it falls short when the algorithm is applied on portrait images. This is due to the nature of portrait images where the face of a person is very sensitive towards abstraction. We adopted Viola's face detection [6] to detect human face. The method used for generating points on the face is similar to that used in edge detection, except that its boundaries are constrained within the detected face region. The x- and y-gaps between the generated seed points on the detected face are computed as:

$$xFaceGap = \frac{2I_w}{N_t} \tag{3}$$

$$yFaceGap = \frac{2I_h}{N_t} \tag{4}$$

Notably, the values of $xFaceGap$ and $yFaceGap$ are two times larger than the gap values of $xGap$ and $yGap$. We double the gap values on the face to avoid generating too many points on the face as our experiment shows that using the same gap values as xGap and yGap would result in over realistic abstraction. The set of all the facial points is represented by P_{f}. If no face is detected by the face detector, no seed points based on face will be generated. Fig.6 illustrates an example of results with and without face detection.

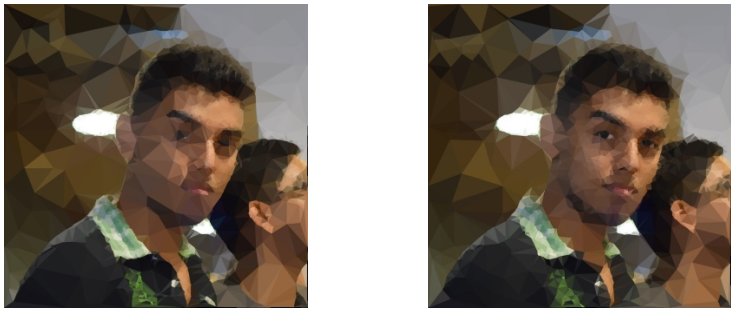

Fig. 6. Comparison of results **(left)** without face detection, and **(right)** with face detection

2.4 Delaunay Triangulation and Colorization

Using the seed points, P, obtained from saliency, edge and face detection as the input, we perform Delaunay triangulation [7] to generate a set of triangular meshes to represent the image. For each triangle generated from Delaunay triangulation, we then color it with the average color of all colors of the pixels contained in the triangle to produce the final low-poly geometric art.

2.5 Performance Optimization

To lower its computational time, we perform several optimization to our algorithm . First, we use a downsampled image as the input to the salient detector. This improves the computational speed by a factor of ten times when an image is downsampled from 1200×800 to 300×200. We then upsample the generated salient mask back to its original resolution for generation of salient points. Notably, the downsampling process is only used for salient mask generation and it does not reduce the resolution of the output abstraction, which is generated based on the input image with the original resolution. We performed numerous comparison between the use of upsampled salient mask and the original salient mask in our algorithm and found that the quality of the resulting low-poly abstraction are comparable.

We also applied parallel processing to the triangulation algorithm by adopting OpenMP [8]. It allows us to fully utilize the CPU processing power by distributing the processing load onto multiple CPUs. This optimization further improved the speed performance by 50%.

3 Experiments and Results

From the results of Pic2Geom illustrated in Fig. 1 and Fig. 7, we can observe that our algorithm is able to produce perceptually accurate abstraction, where denser triangles are use to represent the important area of an image. To evaluate the computational efficiency and quality of our approach, we conducted a quantitative evaluation to evaluate the speed of our algorithm and a qualitative user study to evaluate the quality of our geometric abstraction results respectively. A summary of the results of both evaluations are depicted in Table 1.

Table 1. Summary of quantitative and qualitative results

Method	Speed (s)	User preference (%)
Gai et al [1]	4.791	-
Polyvia [2]	0.247	4.23
DMesh [9]	-	15.77
Pic2Geom	0.476	80.00

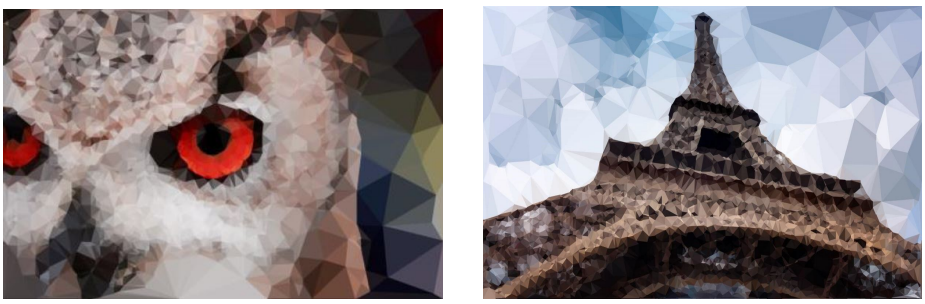

Fig. 7. Example results of Pic2Geom.

3.1 Quantitative User Study

To evaluate the computational speed of our algorithm, we compare our proposed algorithm, Pic2Geom against Zhang et al's [2] (Polyvia) and Gai et al's [1] algorithm. The computational time for Polyvia and Gai et al's method are obtained from the user study conducted by Zhang et al [2] on the Mickey Mouse image illustrated in Fig.8. We use the same computer specification used in Zhang et al's [2] user study, which is a computer with an Intel Core i7-3770 3.4GHz and 8GB RAM, to obtain the computational time for Pic2Geom. The speed comparison depicted in Table 1 shows that the computation time of Pic2Geom is only 2 times slower than Polyvia but 10 times faster than Gai et al's algorithm. From the visual comparison of the three methods in Fig.8, it is obvious that Pic2Geom and Gai et al's approaches can preserve the important features better to produce a more visually appealing abstraction compared to Polyvia. This result shows that our proposed algorithm, Pic2Geom can achieve visual results that are comparable to Gai et al's approach but with significantly less computation time.

(a) Pic2Geom (b) Gai et al [1] (c) Zhang et al [2]

Fig. 8. Visual comparison with state-of-the-arts low poly rendering methods.

3.2 Qualitative User Study

For the qualitative user study, we could not benchmark against Gai et al [1] method because their source code is not publicly available. Thus, we compare our method against Polyvia [2] and DMesh [9]. DMesh is a commercially available software, in which its algorithm is not disclosed. From the results of DMesh, we believe some salient region detection techniques are applied to generate denser triangles on the important area. Comparatively, DMesh produces perceptually more accurate results than Polyvia. We use a set of 15 images for this user study. For each input image, we generate the results for Pic2Geom, Polyvia and DMesh using the default parameters of the three algorithms. Theoretically, it would be desirable to conduct the evaluation using results with the same number of triangles used for the abstraction for all three approaches. However, in practical, it is difficult to obtain the same number of triangles in the results for each approach because given the same user parameter, the number of triangles generated to represent the image abstraction may differ depending on the content of the input images. From the visual comparison of the three approaches in Fig.9, we can observe that the triangles used to represent the abstraction differs in terms of the triangle size and distribution but not so much in terms of the numbers. Thus, we opt to use the default parameters for the user study as the default parameters are determined by each researcher experimentally to produce relatively appealing results for their automatic approach.

The user study is conducted online and we recruited a total of 52 respondents. During the user study, the results of the three methods are displayed in a random order and subjects are instructed to select the most appealing abstraction image out of the three results. In total, we collected a total of 780 votes.

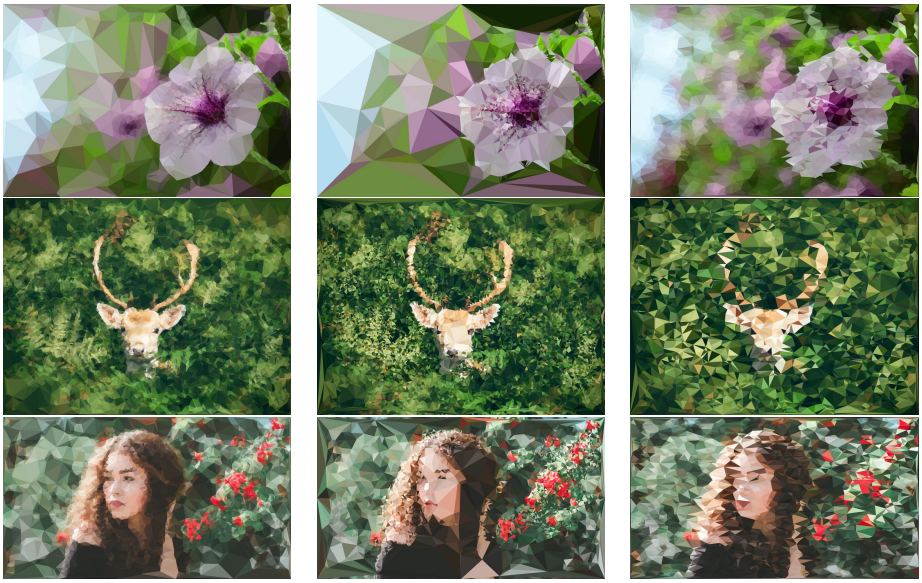

Fig. 9. Comparison of results; **(left)** Pic2Geom, **(middle)** DMesh and **(right)** Polyvia.

From the results of the user study depicted in Table 1, Pic2Geom results are highly preferred over the results of Polyvia and DMesh, with over 80% of the 780 votes goes to Pic2Geom. Interestingly, from the analysis of the results for each image illustrated in Fig.10, Pic2Geom gathered the majority votes for all the 15 images and outperformed Polyvia and DMesh. This is very likely due to the lack of accuracy in detecting the important areas in the Polyvia and DMesh methods.

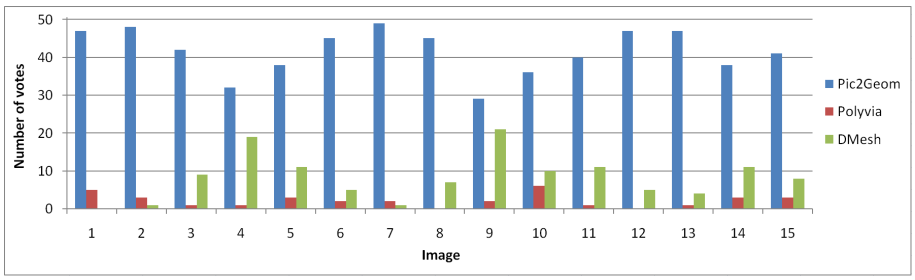

Fig. 10. Results of qualitative study.

The visual comparison of the results in Fig.9 shows that Pic2Geom can distinguish the photo subject more accurately compared to Polyvia and DMesh, particularly for the images of the deer and the girl. DMesh seems to confuse the

textured background with the photo subject and thus produced some very dense triangles in some area of the background. On the other hand, results of Polyvia illustrate a rather uniform distribution of triangles that do not give much focus to the photo subject. These results show that the subjects have a preference for perceptually more accurate abstraction.

4 Conclusion

In this paper, we presented a fast rendering algorithm, Pic2Geom, that automates the low poly rendering to produce good quality geometric abstraction at low computational cost. We were able to achieve perceptually accurate abstraction by using our subject detection techniques that include saliency detection, edge detection, and face detection. We apply several speed optimization to our algorithm to reduce its computational time such as downsampling the input for saliency detection and parallel processing. In our future work, we intend to explore the possibility of incorporating learning-based strategies in pursuit of category-aware geometric abstraction.

References

1. Gai, M., Wang, G.: Artistic Low Poly rendering for images. Visual Computer, 32(4), 491–500 (2016)
2. Zhang, W., Xiao, S., Shi, X.: Low-poly style image and video processing. In: Systems, Signals and Image Processing (IWSSIP) 2015, pp. 97–100. IEEE (2015)
3. Perazzi, F., Krähenbühl, P., Pritch, Y., Hornung, A.: Saliency filters: Contrast based filtering for salient region detection. In: Computer Vision and Pattern Recognition (CVPR) 2012, pp. 733–740. IEEE (2012)
4. Tomasi, C., Manduchi, R.: Bilateral filtering for gray and color images. In: 6th IEEE International Conference on Computer Vision, pp. 839–846. IEEE (1998)
5. Canny, J.: A computational approach to edge detection. IEEE Trans. on Pattern Analysis and Machine Intelligence, 8(6), 679–698 (1986)
6. Viola, P., Jones, M.: Rapid object detection using a boosted cascade of simple features. In: Computer Vision and Pattern Recognition 2001, pp. I-I. IEEE (2001)
7. Delaunay, B.: Sur la sphere vide. Bulletin de l'Acadmie des Sciences de l'URSS, Classe des sciences mathmatiques et naturelles, 6, 793–800 (1934)
8. OpenMP Application Program Interface Version 2.0, `http://www.openmp.org/wp-content/uploads/cspec20.pdf`
9. DMesh, Triangulation Image Generator, `http://dmesh.thedofl.com`
10. Kyprianidis, J. E., Kang, H., Döllner, J.: Image and video abstraction by anisotropic Kuwahara filtering. Computer Graphics Forum, 28(7), 1955–1963. Blackwell Publishing Ltd. (2009)
11. Gerstner, T., DeCarlo, D., Alexa, M., Finkelstein, A., Gingold, Y., Nealen, A.: Pixelated image abstraction. In: Proceedings of the Symposium on Non-Photorealistic Animation and Rendering, pp. 29–36. Eurographics Association (2012)